

Week 10: Express Basics, RESTful API and Socket.io

ผศ. ดร. เกี๊จแกว ฐเนศวร

kejkaew.tha@kmutt.ac.th

ตารางเรียน

สัปดาห์	วันที่	หัวข้อ/รายละเอียด	Assignment (ผู้ตรวจ)	ส่ง Assignment
1	4/8/2569	Introduction to Web Development & HTML5	Assignment 1 (KT,KN)	1
2	11/8/2569	CSS3 Fundamentals	Assignment 2 (KT, KN)	1,2
3	18/8/2569	Advanced CSS & UI Layouts	Assignment 3 (KT)	1,2,3
4	25/8/2569	Responsive Web Design	Assignment 4 (KT, KN)	2,3,4
5	1/9/2569	JavaScript 1: Variables, Data Types, DOM & Event Handling	Assignment 5 (KT, KN)	3,4,5
6	8/9/2569	JavaScript 2: Arrays, Objects & Logic Control	Assignment 6 (KT)	4,5,6
7	15/9/2569	JavaScript 3: DOM Manipulation	Assignment 7 (KT, KN)	5,6,7
8	22/9/2569	Asynchronous JS, AJAX & JSON Data Flow	Assignment 8 (KT, KN)	6,7,8
9	29/9/2569	Introduction to Node.js	Assignment 9 (KT)	7,8,9
10	6/10/2569	Express Basics, RESTful API and Socket.io	Assignment 10 (KT, KN)	8,9,10
	13/10/2569	หยุด		
	20/10/2569	หยุด		
	27/10/2569	หยุด		
11	3/11/2569	Microservices Architecture Concept & Hands-on	Assignment 11 (KT, KN)	9,10,11
12	10/11/2569	Database Integration (MySQL / Firebase)	Assignment 12 (KT, KN)	10,11,12
13	17/11/2569			11,12
14	24/11/2569	ส่งโปรเจค (9:00-16:00)	KT	12
15	12/2569	สอบปลายภาค ในตาราง		

Web Programming Project: Web application

- Web application เป็นเหมือนแอปพลิเคชัน ที่ถูกพัฒนามาให้แสดงผลผ่าน web browser
- คะแนน 30 คะแนน
- กลุ่มละไม่เกิน 3 คน (จับกลุ่มกับเพื่อนต่าง section ได้)

-
- เลือกทำ 1 อย่างจากหัวข้อต่อไปนี้
 - Social media platform
 - Browser game platform
 - Online shopping platform
 - สามารถใช้ CSS, JavaScript, Backend framework ได้
 - ไม่ให้ใช้ web builder หรือ CMS ต่าง ๆ เช่น word press, strapi
 - ไม่ใช่ Unity
 - ใช้ AI ช่วยบางจุดได้ แต่ต้องเข้าใจว่า AI ทำอะไร
 - ส่งรายชื่อ พร้อมเขียน project plan ว่าเราจะทำอะไร พร้อมแผนคร่าว ๆ ส่งวันที่ 30 ต.ค. 2569

รายละเอียด

- Social media platform: สิ่งที่ต้องมีเป็นอย่างน้อย
 - Login page
 - Register page
 - Profile page ของผู้ใ้
 - Feed หรือ Dashboard แสดงข้อความต่างๆ ที่คนใน social media post
 - ผู้ใ้สามารถ post ข้อความ และแสดงใน feed ได้ กด like กด love post ของคนอื่นได้
 - Live chat or stream
 - 1 database

-
- Online shopping platform: สิ่งที่ต้องมีเป็นอย่างน้อย
 - Login page
 - Register page
 - มีหน้า Home แสดงสินค้าต่างๆ ที่กำลังมีขาย
 - ผู้ใช้เป็นผู้ได้ทั้งคนซื้อและคนขาย
 - ผู้ใช้สามารถลงสินค้าใหม่เพื่อขายหรือซื้อสินค้าได้
 - มีหน้า checkout แสดงราคาของทั้งหมดที่ซื้อ
 - Live chat or stream
 - 1 database

-
- Browser game: สิ่งที่ต้องมีเป็นอย่างน้อย
 - Login page
 - Register page
 - Game page (หาเกมที่มีอยู่แล้วได้)
 - Leader board page
 - ต้องกด Like/Love และ comment ใน leader board ได้
 - มี Live chat or stream
 - มีหน้า profile ของผู้ใช้
 - 1 database

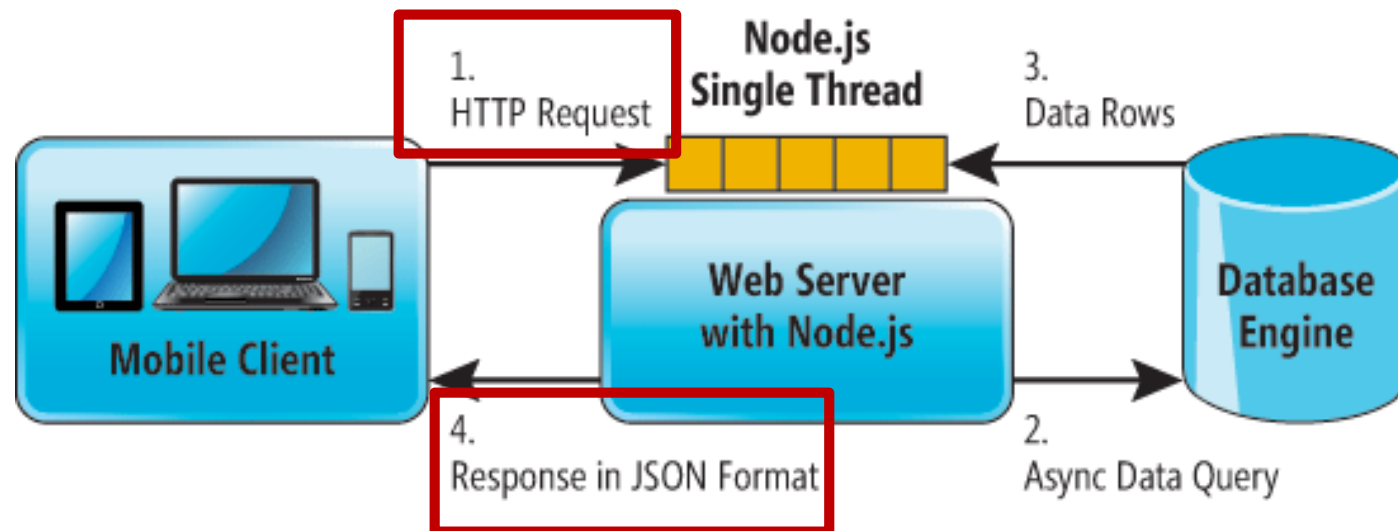
เรื่องที่เรียนไปสัปดาห์ที่แล้ว

- Node.js
- http module
- File system module
- Promises
- Async / Await (Backend)

เรื่องที่จะเรียนวันนี้

- Express module (Backend)
- Fetch API (Frontend)
- Body-Parser (Frontend-Backend)
- Socket.io (Frontend -Backend)
- Assignment 10

ทบทวน



- Client – Server relationship

NPM

- Node Package Manager
- NPM คือ package manager สำหรับ Node.js packages หรือ modules.

วิธีการ install

`npm install <Module Name>`

- ตัวอย่างเช่น install express framework

`npm install express`

Nodemon module

- ใช้สำหรับรัน server ได้เหมือนกับคำสั่ง node
- แต่ข้อดีของ module นี้คือ จะ restart server ได้เองเมื่อ server file มีการเปลี่ยนแปลง

- Installation

npm install -g nodemon

- วิธีการเรียกใช้

nodemon server.js

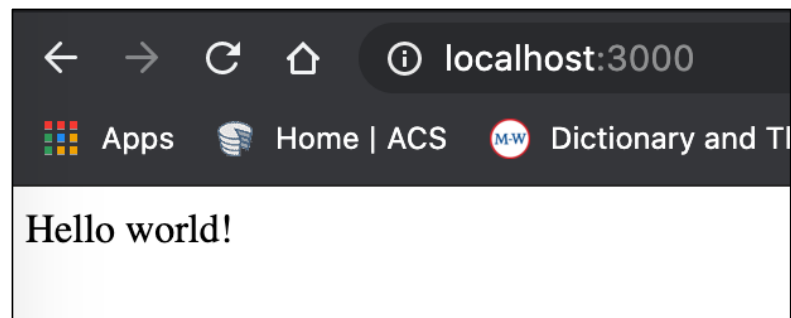
Express Framework

Express คือ Node.js web application framework ที่ใช้สำหรับ พัฒนา web และ mobile applications.

- ทำให้พัฒนา web applications ได้ง่ายและไวขึ้น
- ตัวอย่างที่ Express สามารถทำได้
 - สร้าง middlewares สำหรับจัดการกับ HTTP Requests.
 - กำหนด routing เพื่อทำงานบางอย่างได้ โดยใช้ร่วมกับ HTTP Method และ URL.
 - สามารถ render dynamic HTML Pages ได้ โดยการกำหนด arguments

Express: hello world (ใน Node-2)

```
1  const express = require('express');
2  const app = express();
3  const hostname = 'localhost';
4  const port = 3000;
5
6  app.get('/', (req, res) => {
7    res.send("Hello world!");
8  });
9
10 app.listen(port, hostname, () => {
11   console.log(`Server running at http://${hostname}:${port}/`);
12 });
```



Output

Routing

- Express framework สามารถจัดการ route (เส้นทางการทำงาน, event) ต่างๆ ของ web application ได้ โดยใช้

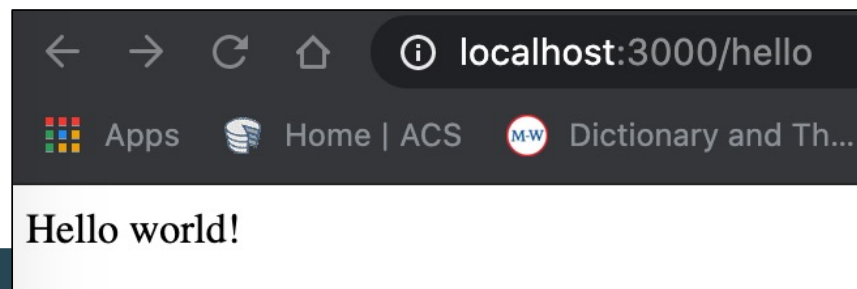
`app.method(path, callback)`

- Method สามารถเป็น get, post, use เป็นต้น
- path คือ route ที่ต้องการให้ทำงานบางอย่าง

ตัวอย่าง

```
14 //routing
15 const express = require('express');
16 const app = express();
17 const hostname = 'localhost';
18 const port = 3000;
19
20 app.get('/hello', (req, res) => {
21   res.send("Hello world!");
22 });
23
24 app.listen(port, hostname, () => {
25   console.log(`Server running at http://${hostname}:${port}/`);
26 });
```

Output



Middleware

- ตัวกลางที่ไว้จัดการ request และ response
- functions ที่สามารถเข้าถึง request object (req), response object (res), และ middleware function ถัดไป (next function) ของ web application
- Middleware functions เหล่านี้ ใช้สำหรับ แก้ไข req และ res objects เช่น กำหนดประเภทของข้อมูลสำหรับการส่ง
- `app.use(callback function)`

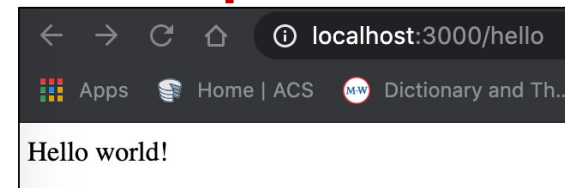
ตัวอย่าง

```
28 // middleware
29 const express = require('express');
30 const app = express();
31 const hostname = 'localhost';
32 const port = 3000;
33
34 app.use((req, res, next) => {
35   console.log("A request for things received at " + Date.now());
36   next();
37 })
38
39 app.get('/hello', (req, res) => {
40   res.send("Hello World!");
41 });
42
43 app.listen(port, hostname, () => {
44   console.log(`Server running at http://${hostname}:${port}/`);
45 });
```

Output: server

```
Server running at http://localhost:3000/
A request for things received at 1617008777380
```

Output: html



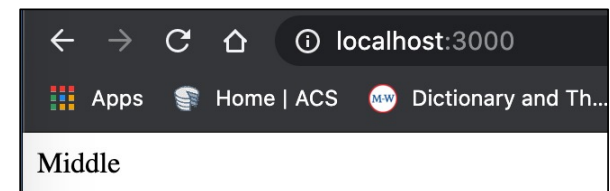
Order of Middleware Calls

- ลำดับการเขียน middleware และ route handler มีความสำคัญ
- โดยโปรแกรมจะทำงานตามลำดับที่เขียนไว้

ตัวอย่าง

```
47 const express = require('express');
48 const app = express();
49 const hostname = 'localhost';
50 const port = 3000;
51
52 app.use((req, res, next) => {
53   console.log("Start");
54   next();
55 })
56
57 app.get('/', (req, res, next) => {
58   res.send("Middle");
59   next();
60 });
61
62 app.get('/', (req, res) => {
63   console.log("End");
64 });
65
66 app.listen(port, hostname, () => {
67   console.log(`Server running at http://\${hostname}:\${port}/`);
68 });
```

Output: html



Output: server

```
Server running at http://localhost:3000/
Start
End
```

Fetch API

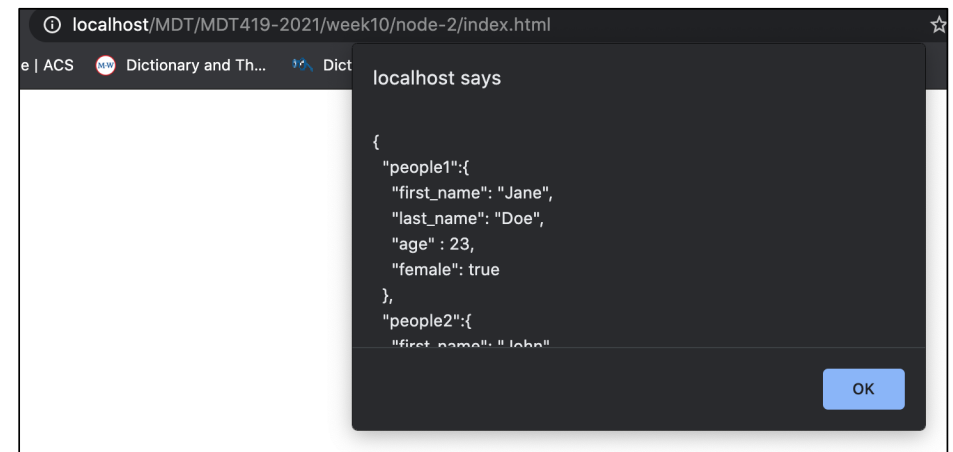
- fetch() มีการทำงานคล้ายกับ XMLHttpRequest (XHR).
- แต่สิ่งที่แตกต่างคือ Fetch API มีการใช้ Promises ทำให้หลีกเลี่ยงการเกิด callback hell ได้
- แต่ AJAX ยังสามารถเกิด callback hell ได้
- fetch function รันบน JavaScript ผ่านทาง browser

บททวน Promises

- Promises เป็นวิธีที่ใช้จัดการกับ asynchronous code เพื่อไม่ให้เกิด callback hell
- เมื่อ promise ถูกเรียกใช้ มันจะอยู่ในสถานะ pending state ซึ่งหมายความว่า callback ทุกตัวที่มี promise ยังทำงานอยู่ และเมื่อตัว callback ก่อนหน้าทำงานสำเร็จ (resolve) แล้ว callback ตัวนั้นก็จะเริ่มทำงานต่อ
- ดังนั้นในการสร้าง promise สุดท้ายแล้วจะเกิดได้สองสถานะ คือ resolved state หรือ rejected state ไปเรื่อยๆ จนจบ chain ของ promise

XHR vs. Fetch: XHR (AJAX) ใน express-fetch/ script.js

```
2 window.onload = pageLoad;
3 function pageLoad(){
4     var xhr = new XMLHttpRequest();
5     xhr.open("GET", "data_2.json",true);
6     xhr.onload = function() {
7         alert(xhr.responseText);
8     };
9     xhr.onerror = function() {
10        alert("ERROR!");
11    };
12    xhr.send();
13 }
```



Output: html

XHR vs. Fetch : fetch (ใน express-fetch/script.js)

```
16 // Fetch, run on localhost
17 const readLog = (async () => {
18     try {
19         const response = await fetch("data_2.json");
20         const data = await response.json();
21         alert(JSON.stringify(data));
22         console.log(data);
23     } catch (err) {
24         console.log(err);
25     }
26 })
27
28 readLog();
```

Output: html

Fetch and Node.js

- Fetch function รันบน client browser
 - ส่ง (request) ข้อมูลไปให้ server และรอรับ (response) ข้อมูลที่ส่งกลับมา
- Node.js รันบน server
 - โดยเราจะใช้ express module ในการรับ request จาก client และส่ง response กลับ

การใช้ Fetch API

- การใช้ Fetch API เริ่มจากเรียกใช้ fetch method และมี parameter คือ url
`fetch(url)`
- Fetch defaults คือ GET requests แต่จะใช้ POST ก็ได้
- `fetch()` method จะทำการเรียกใช้ promise method ดังนั้น เราต้องใช้ `async` and `await` ในการใช้ fetch เหมือนที่เคยทำตอนอ่านไฟล์ (week 9)

การใช้ response ของ Fetch API

- ต้องสร้าง function ที่รับ response argument โดย
 - response ที่เป็น string
 - จะต้องสร้าง resolve promise จาก response.text() โดยสร้าง function ใน then() ที่รับ argument 1 ตัว และ argument ตัวนั้น จะเป็น response ที่เราต้องการ
 - response ที่เป็น JSON
 - จะต้องสร้าง resolve promise จาก response.json() โดยสร้าง function ใน then() ที่รับ argument 1 ตัว และ argument ตัวนั้น จะเป็น response ที่เราต้องการ

ตัวอย่าง fetch ใน (client-server folder)

- ใน client-server folder -> index.js ยังไม่ต้องรัน

```
1 // ตัวอย่าง fetch
2 const sendMsg = (async () => {
3     try {
4         const msg = await fetch("/message");
5         const data = await msg.text();
6         console.log(data);
7     } catch (error) {
8         console.log(error);
9     }
10 })
```

Serving static files in Express

- Node.js ไม่รู้ว่า client content (html, JS, CSS) อยู่ที่ไหน ไม่รู้ว่าจะรอ request จาก client ไหน
- เราต้องบอกว่า server ว่า client content อยู่ใน directory ไດ โดยใช้ express.static method

`express.static(root, [options])`

-
- ใช้ Middleware function เข้ามาช่วยในการจัดการ request และ response
 - `__dirname` เป็น global object ที่เก็บชื่อ directory ที่รัน node.js อยู่
 - เช่น ถ้ารัน node ที่ `/home/user/env` ตัว `__dirname` จะเป็น `/home/user/env`
ดังนั้นจะได้

- `app.use(express.static(__dirname));`
- เป็นการบอกว่า request ที่เข้ามาจะต้องอยู่ใน directory เดียวกันกับ server นะ
ถึงจะทำงานต่อได้

ตัวอย่าง server.js

```
2 const express = require('express');
3 const app = express();
4 const hostname = 'localhost';
5 const port = 3000;
6 const bodyParser = require('body-parser');
7
8 app.use(express.static(__dirname));
9
10 app.get('/message', (req, res) => {
11     console.log("test");
12     res.send("Hello world!");
13 });
14
15 app.listen(port, hostname, () => {
16     console.log(`Server running at http://$
17     {hostname}:${port}/`);
18 });
```

ลองรัน: **http://localhost:3000**

ลองรัน: **http://localhost:3000/message**

Output: server

```
(base) Kejkaews-MacBook-Pro:client-server Kejkaew$ node server.js
Server running at http://localhost:3000/
test
```

Output: html

```
Navigated to http://localhost:3000/
Hello world!
▶ Fetch finished loading: GET "http://localhost:3000/message".
▶ XHR finished loading: GET "chrome-extension://mgijmajocgfcbeboacabfgobmjgjcoja/content.min.css".
```

ผลที่ได้แตกต่างกันอย่างไร

Fetch: POST

- Fetch defaults คือ GET requests แต่จะใช้ POST ก็ได้ เช่น เมื่อต้องการส่ง JSON data ไปให้ server ซึ่งสามารถทำได้โดยกำหนด
 - method เป็น POST
 - body เป็น JSON data
 - headers เป็น {'Accept': 'application/json', 'Content-Type': 'application/json'}

ตัวอย่าง fetch: POST

```
15 const sendMsg = (async (msg) => {  
16     let response = await fetch("/message", {  
17         method: "POST",  
18         headers: {  
19             'Accept': 'application/json',  
20             'Content-Type': 'application/json'  
21         },  
22         body: JSON.stringify({  
23             name: "John",  
24             age: 60  
25         })  
26     });  
27     let content = await response.json();  
28     console.log(content);  
29 });  
30  
31 window.onload = sendMsg;
```

Body-parser middleware

- เนื่องจาก การส่ง input จาก user นั้น ตัว input สามารถเป็นได้หลากหลายแบบ เช่น string, json data etc. เราอาจจะต้องเช็คก่อนว่า input ที่ส่งมาเป็นไปตาม format ที่เราต้องการหรือไม่
- โดยการใช้ body-parser

Install body-parser via terminal

```
npm install body-parser
```

-
- เริ่มต้นจากการทำการ import body-parser

```
const bodyParser = require('body-parser');
```

- จะใช้ body-parser ร่วมกับ middleware เพื่อจัดการกับ request ที่เป็น JSON data โดยใช้

```
bodyParser.json([options])
```

- Returns middleware ที่มีรูปแบบเป็น JSON และจัดการกับ request ที่มี content-type เป็น JSON data ให้เป็น JS object

```
app.use(bodyParser.json());
```

bodyParser.urlencoded([options])

- Returns middleware ที่มีรูปแบบเป็น object จาก url encoded bodies
- ใช้สำหรับกำหนด body object ใหม่
- ใน options สามารถเพิ่ม argument ได้ และใน extended ถ้า value เป็น true จะบอกว่า object ใหม่นั้น มี value เป็นประเภทอะไรก็ได้ แต่ถ้าเป็น false ค่า value จะเป็น string หรือ array

```
app.use(bodyParser.urlencoded({extended: false}));
```

ตัวอย่าง server สำหรับ POST

```
20 const express = require('express');
21 const app = express();
22 const hostname = 'localhost';
23 const port = 3000;
24 const bodyParser = require('body-parser');
25
26 app.use(express.static(__dirname));
27 app.use(bodyParser.json());
28 app.use(bodyParser.urlencoded({extended: false}));
29
30 app.post('/message', (req, res) => {
31   console.log(req.body)
32   const outMsg = req.body
33   outMsg["lastname"] = "Doe";
34   console.log(outMsg);
35   res.json(outMsg);
36 })
37
38 app.listen(port, hostname, () => {
39   console.log(`Server running at http://\${hostname}:\${port}/`);
40 });
```

Output: server

```
Server running at http://localhost:3000/
{ name: 'John', age: 60 }
{ name: 'John', age: 60, lastname: 'Doe' }
```

Output: html

```
Navigated to http://localhost:3000/
▶ Fetch finished loading: POST "http://localhost:3000/message".
▶ {name: "John", age: 60, lastname: "Doe"}
```

Socket.io

- ข้อจำกัดของ HTTP / REST API เดิม (Request-Response Model):
 - Client ต้องเป็นฝ่ายเริ่มร้องขอ (Request) ไปหา Server ก่อนเสมอ Server ถึงจะตอบกลับ (Response) ได้
 - ถ้ามีข้อมูลใหม่เกิดขึ้นที่ Server ฝ่าย Client จะไม่มีทางรู้เลย จนกว่าจะกดรีเฟรชหน้าเว็บ หรือใช้ระบบยิงถามซ้ำๆ (Polling) ซึ่งเปลืองทรัพยากรมาก
- ทางออกด้วย WebSockets:
 - เทคโนโลยีที่เข้ามาช่วยเปิดท่อการสื่อสารแบบสองทาง (Bi-directional / Full-Duplex) ค้างไว้ตลอดเวลา

Socket.io คืออะไร?

เป็น JavaScript Library ยอดนิยมที่ครอบทับ WebSocket อีกชั้นหนึ่ง เพื่อช่วยจัดการเรื่องที่ซับซ้อนให้อัตโนมัติ เช่น การเชื่อมต่อใหม่เมื่อเน็ตหลุด (Auto-reconnection) และระบบการกระจายข้อความแบบแบ่งห้อง (Rooms & Namespaces)

หลักการทำงานของ Socket.io

- `socket.emit('event-name', data)` (การยิงสัญญาณ/ส่งข้อมูล):
 - เหมือนการส่งจดหมายหรือการตะโกน โดยระบุ "ชื่อเรื่อง (Event Name)" และ "เนื้อหา (Data)" ที่ต้องการส่ง
- `socket.on('event-name', callback)` (การรอรับฟัง):
 - เหมือนการตั้งเสาารับสัญญาณ เมื่อได้ยินชื่อ Event ที่ตรงกัน จะนำข้อมูลไปทำคำสั่งในฟังก์ชันทันที
- **ทิศทางการกระจายข้อความ (Broadcast Types) ของ Server:**
 - `socket.emit(...)` = ส่งกลับไปหา Client คนที่ติดต่อเข้ามาคนเดียว
 - `socket.broadcast.emit(...)` = ส่งหาทุกคนในระบบ ยกเว้น ตัวเอง
 - `io.emit(...)` = ส่งหาทุกคนในระบบ **รวมทั้ง** ตัวเองด้วย
 - `io.to('room-id').emit(...)` = ส่งหาเฉพาะคนที่อยู่ในห้องนั้นๆ

เริ่มทำการเชื่อมต่อ

- Client : ต้อง import: `const socket = io();` ใน `script.js` และมีการเรียก `<script src="/socket.io/socket.io.js"></script>` ใน HTML
- Server: import ทั้ง HTTP, socket.io and Express module
- สร้าง connection ใน server โดยใช้ `io.on('connection', (socket) => { ... })` คือ Event Listener ฟังก์ชัน Server ของ Socket.io ที่คอยตรวจจับและดักฟังว่า "มี Client (ผู้ใช้งาน/เบราว์เซอร์) รายใหม่เชื่อมต่อเข้ามายังเซิร์ฟเวอร์สำเร็จหรือไม่"

```

53 // รับ ID ของตัวเองจาก Server
54 socket.on('your_id', (data) => {
55     myId = data.id;
56     updateStatusBadge();
57 });
58
59 // รับข้อความทั่วไป (ที่ส่งมาจาก socket.emit, socket.
    broadcast.emit, io.emit, io.to.emit)
60 socket.on('message_received', (data) => {
61     addLog(data);
62 });
63
64 // รับการแจ้งเตือนการเข้าห้อง
65 socket.on('room_notification', (data) => {
66     addSystemLog(`🔔 [ห้อง ${data.room}] ${data.message}`);
67 });

```

Script.js

```

17 io.on('connection', (socket) => {
18     console.log(`[+] Client เชื่อมต่อแล้ว: ${socket.id}`);
19
20     // ส่ง ID ของตัวเองกลับไปให้ Client ทราบ
21     // socket.emit('event-name', data) -> ส่งกลับไปหา Client คนนี้คนเดียว
22     socket.emit('your_id', { id: socket.id });
23
24     // -----
25     // 1) ตัวอย่าง: socket.emit(...)
26     // "ส่งกลับไปหา Client คนที่ติดต่อเข้ามาคนเดียว"
27     // -----
28     socket.on('send_only_me', (data) => {
29         console.log(`[Only Me] จาก ${socket.id}:`, data);
30         // socket.emit = ส่งกลับไปหา Client คนที่ส่งมาเท่านั้น
31         socket.emit('message_received', {
32             type: 'socket.emit',
33             description: 'ส่งหาคุณคนเดียว (Only Me)',
34             senderId: socket.id,
35             text: data.text,
36             time: new Date().toLocaleTimeString()
37         });
38     });

```

Server.js

```

89 // แบบที่ 1: ส่งหาฉันคนเดียว (Only Me)
90 function sendOnlyMe() {
91     const text = messageInput.value.trim() ||
92     'ข้อความส่งหาฉันคนเดียว';
93     // socket.emit('event-name', data)
94     socket.emit('send_only_me', { text: text });
95 }
96 // แบบที่ 2: ส่งหาทุกคน ยกเว้นฉัน (Broadcast)
97 function sendBroadcast() {
98     const text = messageInput.value.trim() || 'ข้อความ
99     Broadcast (ทุกคนยกเว้นฉัน)';
100     // socket.emit('event-name', data)
101     socket.emit('send_broadcast', { text: text });
102 }

```

Script.js

Server.js

```

24 // -----
25 // 1) ตัวอย่าง: socket.emit(...)
26 // "ส่งกลับไปหา Client คนที่ติดต่อเข้ามาคนเดียว"
27 // -----
28 socket.on('send_only_me', (data) => {
29     console.log(`[Only Me] จาก ${socket.id}:`, data);
30     // socket.emit = ส่งกลับไปหา Client คนที่ส่งมาเท่านั้น
31     socket.emit('message_received', {
32         type: 'socket.emit',
33         description: 'ส่งหาคุณคนเดียว (Only Me)',
34         senderId: socket.id,
35         text: data.text,
36         time: new Date().toLocaleTimeString()
37     });
38 });
39
40 // -----
41 // 2) ตัวอย่าง: socket.broadcast.emit(...)
42 // "ส่งหาทุกคนในระบบ ยกเว้น ตัวเอง"
43 // -----
44 socket.on('send_broadcast', (data) => {
45     console.log(`[Broadcast] จาก ${socket.id}:`, data);
46     // socket.broadcast.emit = ส่งหาทุกคน ยกเว้น socket ตัวเอง
47     socket.broadcast.emit('message_received', {
48         type: 'socket.broadcast.emit',
49         description: 'ส่งหาทุกคน ยกเว้นคนส่ง (Broadcast)',
50         senderId: socket.id,
51         text: data.text,
52         time: new Date().toLocaleTimeString()
53     });
54 });

```

```

103 // แบบที่ 3: ส่งหาทุกคน รวมฉันด้วย (To All)
104 function sendToAll() {
105     const text = messageInput.value.trim() || 'ข้อความถึงทุกคน
      (io.emit)';
106     // socket.emit('event-name', data)
107     socket.emit('send_to_all', { text: text });
108 }

```

Script.js

```

56 // -----
57 // 3) ตัวอย่าง: io.emit(...)
58 // "ส่งหาทุกคนในระบบ รวมทั้ง ตัวเองด้วย"
59 // -----
60 socket.on('send_to_all', (data) => {
61     console.log(`[All] จาก ${socket.id}:`, data);
62     // io.emit = ส่งหาทุกคนทั้งหมดในระบบ
63     io.emit('message_received', {
64         type: 'io.emit',
65         description: 'ส่งหาทุกคน รวมตัวเอง (To All)',
66         senderId: socket.id,
67         text: data.text,
68         time: new Date().toLocaleTimeString()
69     });
70 });

```

Server.js

Script.js

```
110 // ขอเข้าร่วมห้อง
111 function joinRoom() {
112     currentRoom = roomSelect.value;
113     // socket.emit('event-name', data)
114     socket.emit('join_room', currentRoom);
115     roomStatus.textContent = `ห้องปัจจุบัน: ${currentRoom}`;
116     roomStatus.style.background = '■#8b5cf6';
117 }
```

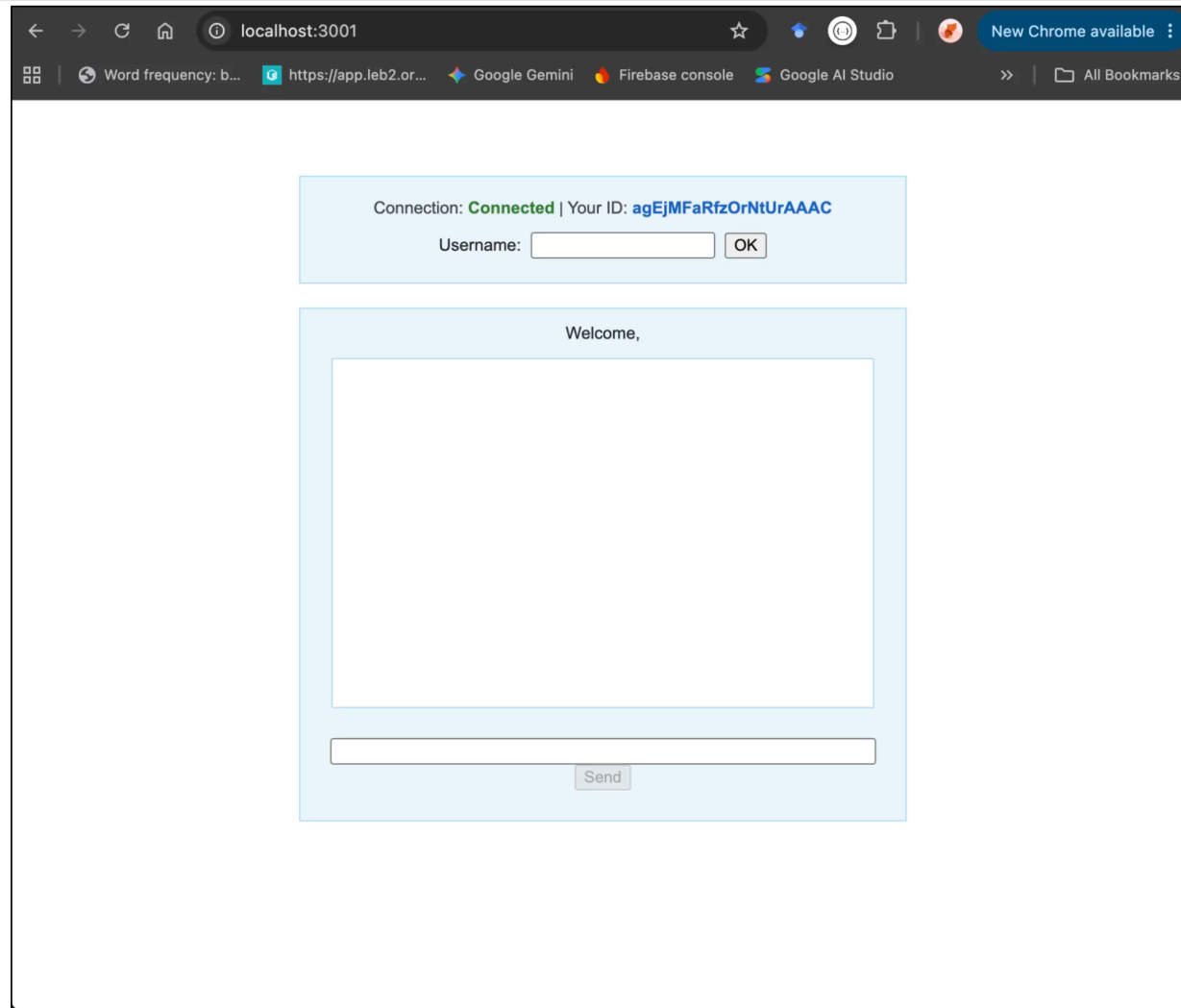
```
119 // แบบที่ 4: ส่งเฉพาะคนที่อยู่ในห้อง (To Room)
120 function sendToRoom() {
121     if (!currentRoom) {
122         alert('กรุณากด "เข้าห้องนี้" ก่อนส่งข้อความเข้าห้องครับ');
123         return;
124     }
125     const text = messageInput.value.trim() ||
126     `ข้อความถึงสมาชิกในห้อง [${currentRoom}]`;
127     // socket.emit('event-name', data)
128     socket.emit('send_room_message', {
129         room: currentRoom,
130         text: text
131     });
132 }
```

Server.js

```
72 // -----
73 // 4) ตัวอย่าง: io.to('room-id').emit(...)
74 // "ส่งหาเฉพาะคนที่อยู่ในห้องนั้นๆ"
75 // -----
76 // เมื่อ Client ขอย้ายเข้าห้อง
77 socket.on('join_room', (roomName) => {
78     // ออกจากห้องเดิมก่อน (ถ้ามี) ยกเว้น default room (socket.id)
79     for (const room of socket.rooms) {
80         if (room !== socket.id) {
81             socket.leave(room);
82             console.log(`[-] ${socket.id} ออกจากห้อง ${room}`);
83         }
84     }
85
86     // เข้าร่วมห้องใหม่
87     socket.join(roomName);
88     console.log(`[+] ${socket.id} เข้าร่วมห้อง: ${roomName}`);
89
90     // แจ้งเตือนคนในห้องนั้น
91     io.to(roomName).emit('room_notification', {
92         room: roomName,
93         message: `สมาชิกใหม่ (${socket.id}) เข้าร่วมห้องแล้ว`
94     });
95 });
96
97 // ส่งข้อความเฉพาะห้องที่เลือก
98 socket.on('send_room_message', (data) => {
99     console.log(`[Room: ${data.room}] จาก ${socket.id}:`, data.text);
100     // io.to('room-id').emit = ส่งหาเฉพาะ client ที่อยู่ในห้องนั้นๆ
101     io.to(data.room).emit('message_received', {
102         type: `io.to('${data.room}').emit`,
103         description: `ส่งเฉพาะคนที่อยู่ในห้อง [${data.room}]`,
104         senderId: socket.id,
105         room: data.room,
106         text: data.text,
107         time: new Date().toLocaleTimeString()
108     });
109 });
```

Assignment 10:

- Download assignment-10.zip file
- เราจะทำการสร้าง chatroom ง่าย ๆ โดยใช้ node.js เริ่มจาก
 - ให้ user กรอกชื่อ
 - อ่าน json data จาก log.json แสดงใน chatroom
 - ให้ user พิมพ์ข้อความที่ต้องการส่ง
 - ทำการ save ข้อความที่ user พิมพ์ใน chatroom ของเราลงใน file ที่มีชื่อว่า log.json
 - ใช้ Async, Await, express และ socket.io
- แก้ไขใน script.js กับ server.js
- ส่งด้วยนะคะ



Week 10: Pop Quiz

- ตอบคำถาม 8 ข้อแบบออนไลน์
- มีเวลา 15 วินาทีในการตอบแต่ละข้อ
- <https://ketnas.github.io/popQuiz/>

